openAG — Entity Model (Generation 0)

Status: Design / seed model. This is Generation 0 — the bootstrap, not a final or closed set (see §12 Growth Frontier). Companion to `2026-10-04-openAG-graph-native-engine-design.md` and `2026-10-04-openAG-pipelines.md`. Date: 2026-10-04

> Revision 2026-10-12 (session ledger fold-in). Propagated the brainstorming-session locks into the gen-0 entity set: - Neural govern replaces the deterministic BudgetGovernor (§4.2 `COMP-BudgetGovernor`, §5.1 `BEH-ROUND.govern`): 4-step symbolic-legality → symbolic-budget → neural-VoI → symbolic-clamp. - Runtime oracle added as a first-class fact-source actor (`ACT-Runtime`, §7.1) — the 4th oracle alongside code-scan, human-dialogue, model-self. - Two promises per Interface (static left-of-Vee + dynamic right-of-Vee) made explicit (§6), with `gap:integration-mismatch` for an ungrounded dynamic promise. - Risk is intrinsic to every promise (P(won't ground) × consequence), collapsed into VoI — not a new entity (§9 note). - B0 seed graph of promised entities noted on `SYS-openAG` (§1) — stakeholder-needs opens one continuous ROUND. - New VoI/economic gap classes added to §10 (`gap:trivial`, `gap:low-confidence`, `gap:integration-mismatch`).

---

0. How to read this doc

Every entity is given as `id · name · description · attribute-promises · key relationships`. Attributes are anchored to the real v3 **`PromiseSpec`** (`attribute, content, owner_entity, delivered_from, verified_by, status, gap_type, provenance`; `promise-graph.schema.json`, `PromiseStatus ∈ {open, delivered, verified, typed-gap}`) extended with two net-new openAG fields:

- **`axis`** — which decomposition axis the promise points along (A1 Abstraction · A2 Realization · A3 Boundary · A4 Governance · A5 Flow · A6 Lifecycle · A7 Evidence).
- **`valence`** — the kind-rule that demanded the promise (from the per-kind valence table, design §4.1).

In the tables below, attribute-promises lists the required promises (absence ⇒ the named gap); each carries an `axis` tag. The base row for every entity is implicit: `id, name, kind, status (ACTIVE|PLANNED|…), intent` + ≥1 A7 grounding or `gap:ungrounded`.

Decomposition depth is non-uniform (SE-correct): atomic leaves stop at L2; non-atomic branches go to L3+.

---

1. System (L0)

id	name	description	attribute-promises (axis)	relationships
`SYS-openAG`	openAG	A graph-native architecture engine that sustains a verified architecture graph of a target system by promise-discharge.	mission/F0 (node-local) · success_criteria (node-local) · top-constraints →`CON-`∗ (A4) · health ← Σ capability health (A1↑ roll-up)	`contains` → CAP-extract, CAP-model, CAP-represent, CAP-learn, CAP-mdd (A1)

F0: "Sustain a verified architecture graph of a target system."

B0 seed graph (stakeholder-needs opening). The bootstrap input to SYS−openAG is a seed graph of promised entities — the System + Actors + top Capabilities, all with OPEN promises and none grounded. B0 is thin, coarse ROUND-seeding: the high-level goal is robust and order-insensitive; fine detail is deferred to the confluent ROUND. A stakeholder need = an OPEN promise (requirement / goal / success_criteria) owned by the voicing Actor. There is no hard B0→B1 boundary — one continuous ROUND, with the human oracle active early and fading as artifacts ground the graph. Termination is the VoI test with human-attention as the budget.

---

2. Capabilities (L1) — 5

id	name	description	attribute-promises (axis)	realized by
CAP−extract	extract	Produce seed nodes + grounded facts from a target system's source via pure AST analysis.	intent · success_criteria · owner · ≥1 realizes (A2, else gap:unrealized) · realization-coverage ← children (A1↑)	COMP SourceScanner, PatternDetector, Seeder
CAP−model	model	Grow the graph to least-fixpoint by discharging open promises along their axes.	intent · success_criteria · owner · ≥1 realizes (A2) · SLO → sub-cap SLOs (A1↓)	COMP ExpansionKernel + model-layer components
CAP−represent	represent	Materialize, project, and render any owned view/artifact from the graph.	intent · success_criteria · owner · ≥1 realizes (A2)	COMP Slicer, ProjectorRegistry, Renderers, Viewer
CAP−learn	learn	Observe outcomes, distill lessons, and tune the rules that drive discharge.	intent · success_criteria · owner · ≥1 realizes (A2)	COMP FeedbackJournals, LessonStore, RuleTuner
CAP−mdd	mdd	Model-driven generation of code + behavioral parity verification against an oracle.	intent · success_criteria · owner · ≥1 realizes (A2)	COMP CodeGenerator, ParityChecker

---

3. Functional hierarchy — L2 sub-capabilities + non-uniform L3+

Atomic leaves (■) stop at L2. Non-atomic branches (▾) decompose to L3. L3 functions are the operator stages (design §3.2) and the §8 algorithms, promoted into the functional hierarchy.

3.1 CAP-extract (3 sub-caps)

id	name	depth	description	L3 functions
CAP-extract.scan	scan	▾	AST-walk the target tree; emit observable facts.	walk-files · parse-AST · extract-imports · extract-routes/endpoints · extract-constants
CAP-extract.detect	detect	▾	Discover architecturally-significant patterns.	pattern-match (http/cli/message/rpc) · cluster-by-affinity
CAP-extract.seed	seed	■	Emit seed nodes (one per module/endpoint) with open promises.	—

3.2 CAP-model (7 sub-caps — the operator)

id	name	depth	description	L3 functions
CAP-model.grow	grow	▾	Pick and expand an open promise into candidate edges/nodes.	select-promise · propose-candidates
CAP-model.ground	ground	▾	Verify candidates against evidence.	gather-evidence · check-grounding · score-confidence
CAP-model.gate	gate	▾	Admit only schema+constraint-valid candidates.	admit-schema · admit-constraints
CAP-model.reconcile	reconcile	▾	Normalize to canonical form (closure operator).	dedup · identity-bind(θ) · roll-up-aggregate · canonicalize
CAP-model.govern	govern	▾	Schedule discharge under budget (critical-path, widening).	build-dep-DAG · compute-critical-path/slack · tier-schedule · budget-widen · batch

id	name	depth	description	L3 functions
CAP-model.remember	remember	▾	Persist decisions, identities, generations.	cache-decision · record-generation
CAP-model.traverse	traverse	▾	Run declarative walks over the graph.	select-nodes · hop · classify · rollup · evidence · emit-cards

3.3 CAP-represent (3 sub-caps)

id	name	depth	description	L3 functions
CAP-represent.slice	slice	▾	Materialize a named curated subset (contract → fragment).	select · close-relationships · curate/redact
CAP-represent.project	project	▾	Turn a materialized slice + ViewSpec into a DiagramSpec.	resolve-projector · stamp-freshness/provenance
CAP-represent.render	render	■	Deterministically render a ProjectedView to bytes (md/html/svg/…).	—

3.4 CAP-learn (3 sub-caps)

id	name	depth	description	L3 functions
CAP-learn.observe	observe	▾	Capture outcomes into append-only journals.	append-gate/drift/test/llm-call · read-journal
CAP-learn.lesson	lesson	■	Distill deduplicated lessons from journals.	—
CAP-learn.tune	tune	■	Adjust expander/scheduler rules from lessons (closes B6).	—

3.5 CAP-mdd (2 sub-caps)

id	name	depth	description	L3 functions
CAP-mdd.generate	generate	▾	Generate code from the model; converge against an oracle.	scaffold · fill-bodies · oracle-loop
CAP-mdd.verify-parity	verify-parity	▾	Run the oracle, diff, score behavioral parity.	run-oracle · diff · score-parity

Non-atomic L2 count: 15 of 18 decompose to L3 (3 atomic leaves: seed, render, lesson, tune — note lesson/tune atomic).

---

4. Components — 26

Attribute rows anchor to schema component $defs (layer, files, kind, intent, goals, failure_modes, trade_offs, dependencies_rationale, verification, owner, evidence, …).

4.1 Substrate layer (extract)

id	name	description	key attribute-promises (axis)	key relationships
COMP-SourceScanner	SourceScanner	AST-walks a target tree; emits the observable-fact inventory.	layer=Substrate (A4) · kind=library · intent · anchor (forked from *pipeline_v3.scan* + *manifest/scanner.py*)	realizes→CAP-extract.scan (A2) · produces→Manifest/ScanReport (A5)
COMP-PatternDetector	PatternDetector	Detects end-points/handlers/clusters (http/cli/message/rpc).	layer=Substrate · intent · dependencies_rationale	realizes→CAP-extract.detect · depends-on→SourceScanner (A3)
COMP-Seeder	Seeder	Emits seed nodes with their kind-valence open promises.	layer=Substrate · intent	realizes→CAP-extract.seed · depends-on→PatternDetector

4.2 Kernel layer (model) — 13 (incl. support SeedStrategy allocation)

id	name	description	key attribute-promises (axis)	key relationships
COMP–ExpansionKernel	ExpansionKernel	Coordinator of the fixpoint; runs B-ROUND to least-fixpoint.	layer=Kernel · intent · failure_modes · gap:budget–exhausted handling	realizes→CAP-model.grow (A2) · depends–on→PromiseEngine, Grounder, GateEvaluator, Reconciler, BudgetGovernor (A3)
COMP–PromiseEngine	PromiseEngine	Computes open promises per node from kind-valence; tracks lifecycle.	intent · valence-table ref · dependen-cies_rationale	realizes→CAP-model.grow · pro–duces→Promise nodes (A5)
COMP–ExpanderRegistry	ExpanderRegistry	Total registry of (kind, axis) → expander (symbolic/neural).	intent · total-registry invariant	realizes→CAP-model.grow · ex–poses→ExpanderRegistry-API (A3)
COMP–Grounder	Grounder	Checks candidate edges/nodes against typed evidence.	intent · evidence-taxonomy (static/observational/executable)	realizes→CAP-model.ground · con–sumes→EvidenceRef (A3)
COMP–GateEvaluator	GateEvaluator	Admits only schema+constraint-valid candidates; emits gaps.	intent · 3 verification classes	realizes→CAP-model.gate · con–sumes→Constraints (A3)
COMP–Reconciler	Reconciler	Closure operator: dedup, identity-bind, roll-up, canonicalize.	intent · confluence-modulo-budget	realizes→CAP-model.reconcile · depends–on→IdentityStore (A3)

id	name	description	key attribute-promises (axis)	key relationships
COMP–BudgetGovernor	BudgetGovernor	Neural govern scheduler: (1) symbolic legality filter (down-only acyclic promise-DAG), (2) symbolic budget read (owns widening/termination), (3) neural prioritization by VoI, (4) symbolic bundle + budget-clamp. Correctness-neutral (govern = accelerator, confluence is guarantor).	intent · internal-only (no interface) · dependencies_rationale · temp-0 for reproducible gap-sets · schedules only down-and-local half (reconcile owns up-flows)	realizes→CAP-model.govern · produces→Job (A5) · depends–on→LLMProviderRouter (neural VoI step, A3)
COMP–ModelGraph	ModelGraph	Pure state — the mutable graph (nodes, edges, promises). Anti-god-object: no logic.	intent · CON–pure–state · NN-compilable (C-N2)	constrained–by→CON-pure-state (A4) · operated-on by kernel
COMP–DecisionCache	DecisionCache	Memoizes expander decisions for incremental re-runs.	intent · internal-only	realizes→CAP-model.remember
COMP–IdentityStore	IdentityStore	Sticky θ-crosswalk; stable IDs across generations.	intent · internal-only · θ_high=0.85/θ_low=0.60 (C-N2)	realizes→CAP-model.remember
COMP–GenerationStore	GenerationStore	Publishes + versions model generations; owns generation lineage.	intent · owner of Generation entities	realizes→CAP-model.remember · produces→Generation (A5)
COMP–WalkEngine	WalkEngine	Runs declarative WalkSpecs (select→…→emit cards).	intent · forked from AMS walks/	realizes→CAP-model.traverse · consumes→WalkSpec (A3)
COMP–SeedStrategy	SeedStrategy	Chooses seeding policy (forward-author vs code-extract).	intent · internal-only (support)	realizes→CAP-extract.seed (A2)

4.3 Projection layer (represent) — 4

id	name	description	key attribute-promises (axis)	key relationships
COMP-Slicer	Slicer	Materializes ModelSlice contracts into closed fragments.	layer=Projection · intent · forked from lifecycle/model_slice_materialize.py	realizes→CAP-represent.slice · consumes→SliceSpec (A3)
COMP-ProjectorRegistry	ProjectorRegistry	Named registry of pure projectors → DiagramSpec.	intent · exposes→ProjectorRegistry-API	realizes→CAP-represent.project · exposes→ProjectorRegistry-API (A3)
COMP-Renderers	Renderers	Deterministic md/html/svg/zip/ai-context/pipeline-html renderers.	intent · byte-determinism	realizes→CAP-represent.render · produces→Artifact (A5)
COMP-Viewer	Viewer	Self-contained HTML lenses (v3 promise-graph, legacy box-arrow, dashboard).	intent · forked from viewers/	realizes→CAP-represent.render · produces→Artifact (A5)

4.4 Adaptation layer (learn) — 3

id	name	description	key attribute-promises (axis)	key relationships
COMP-FeedbackJournals	FeedbackJournals	Append-only JSONL streams (gates, drift, test_results, llm_calls). Owner of Journal entities.	layer=Adaptation · intent · fail-soft	realizes→CAP-learn.observe · produces→Journal (A5)
COMP-LessonStore	LessonStore	Deduplicated lesson store (fixes AMS “lessons overwritten”).	intent · internal-only · append+dedup	realizes→CAP-learn.lesson · produces→Lesson (A5)
COMP-RuleTuner	RuleTuner	Tunes expander/scheduler rules from lessons; re-enters B-ROUND.	intent · closes B6 loop	realizes→CAP-learn.tune · affects→ExpanderRegistry, BudgetGovernor (A4)

4.5 mdd layer — 2

id	name	description	key attribute-promises (axis)	key relationships
COMP–CodeGenerator	CodeGenerator	Generates code from the model; oracle-in-loop convergence.	intent · oracle-obtainable precondition	realizes→CAP-mdd.generate · produces→Artifact(code) (A5)
COMP–ParityChecker	ParityChecker	Runs the oracle, diffs, scores behavioral parity.	intent · parity metric	realizes→CAP-mdd.verify-parity · verifies→behaviors (A7)

4.6 Cross-cutting support — 1

id	name	description	key attribute-promises (axis)	key relationships
COMP–LLMProviderRouter	LLMProvider/Router	Provider-agnostic routing (policy, budget, fallback, bundling).	cross-cutting · intent · exposes→LLMProvider-Protocol	exposes→LLMProvider-Protocol (A3) · uses→external LLM service

Component count = 26 (3 substrate + 13 kernel + 4 projection + 3 adaptation + 2 mdd + 1 cross-cutting). SeedStrategy is counted in the kernel block above as a model-support component.

---

5. Behaviors — B-ROUND + B1–B7 (enumerated)

Each top-level behavior fans into per-stage sub-behaviors. IDs BEH–∗. I/O columns name the owned I/O entities (§8).

5.1 B-ROUND (the shared fixpoint)

id	name	trigger	steps (sub-behaviors)	traces-to
BEH–ROUND	round	B1/B2/B3/B6	govern → grow → ground → reconcile → bind → gate	ExpansionKernel
BEH–ROUND.govern	govern-step	—	legality-filter (symbolic) · budget-read (symbolic) · prioritize-VoI (neural) · bundle+clamp (symbolic)	BudgetGovernor, LLMProvider-Router

id	name	trigger	steps (sub-behaviors)	traces-to
BEH-ROUND.grow	grow-step	—	select · propose	ExpansionKernel, ExpanderRegistry
BEH-ROUND.ground	ground-step	—	gather · check · score	Grounder
BEH-ROUND.reconcile	reconcile-step	—	dedup · identity-bind · roll-up · canonicalize	Reconciler, IdentityStore
BEH-ROUND.bind	bind-step	—	record-generation	GenerationStore
BEH-ROUND.gate	gate-step	—	admit-schema · admit-constraints · emit-gaps	GateEvaluator

5.2 Top-level behaviors

id	name	trigger (actor/behavior)	outline	produces (owned I/O)	traces-to
BEH-B1	Extract	Modeler / Target System	scan → derive-promises → cluster → link → containerize → verify → publish (forks pipeline_v3) → B-ROUND	Generation, Manifest	SourceScanner…GenerationSto
BEH-B2	Author	Modeler	note → save → author → apply (CAP-DF ceremony) → B-ROUND	Note, WorkOrder, Job, Proposal	WalkEngine, LLMProvider-Router
BEH-B3	Re-extract	Target System (change)	re-scan → diff → identity-bind → B-ROUND	Generation, drift Journal	SourceScanner, IdentityStore
BEH-B4	Represent	Modeler / Downstream	slice → materialize → project → render → rebuild	Artifact (+provenance sidecar)	Slicer, Projec-torRegistry, Renderers, Viewer
BEH-B5	Generate	Modeler / arch-agent	scaffold → fill-bodies → oracle-loop → verify-parity	Artifact(code), test_results Journal	CodeGenerator, ParityChecker
BEH-B6	Learn	Auditor / self	observe → lesson → tune → re-enter B-ROUND	Lesson, Journals	FeedbackJournals, LessonStore, RuleTuner

id	name	trigger (actor/behavior)	outline	produces (owned I/O)	traces-to
BEH-B7	Gate	Auditor	run gate checks (= B-ROUND.gate standalone)	gates Journal, GateEvent	GateEvaluator

Behavior count ≈ 33: 7 B-ROUND rows + 7 top-level + ~19 fan-out sub-behaviors (scan/derive/cluster/link/containerize/verify/publish for B1; note/save/author/apply for B2; etc.). Full fan-out enumerated in the pipelines companion doc.

---

6. Interfaces — 13

id	name	subkind	exposer → consumer	schema/contract
IF-ExpanderRegistry	ExpanderRegistry-API	library	ExpanderRegistry → ExpansionKernel	register(kind,axis,expander) / resolve
IF-ProjectorRegistry	ProjectorRegistry-API	library	ProjectorRegistry → project step	register(name) / resolve
IF-LLMProvider	LLMProvider-Protocol	rpc	LLMProviderRouter → expanders	complete/stream/structured/t
IF-Patch	Patch/Proposal-schema	library	author step → apply	ModelPatch (add/remove/replace)
IF-ScanReport	ScanReport/Manifest	library	SourceScanner → Seeder	observable-fact inventory
IF-EvidenceRef	EvidenceRef (typed)	library	Grounder ↔ evidence	static/observational/executable
IF-PromiseSpec	PromiseSpec/valence-table	library	PromiseEngine → kernel	v3 PromiseSpec + axis/valence
IF-WalkSpec	WalkSpec	library	author → WalkEngine	select/traverse/classify/rollup/evider
IF-SliceSpec	SliceSpec	library	Modeler → Slicer	ModelSlice contract
IF-ViewSpec	ViewSpec	library	Modeler → project	projector + output kind
IF-Journal	Journal-append-API	library	producers → FeedbackJournals	append-only JSONL
IF-GraphExport	nn-compilable-graph-export	library	ModelGraph → arch-agent/N2	lossless MultiDiGraph (C-N2)
IF-CLI	CLI/MCP-surface	cli	Modeler/Auditor → openAG	commands + MCP tools

Honest internal-only (no interface): BudgetGovernor, DecisionCache, IdentityStore, SeedStrategy, LessonStore.

Two promises per Interface (left-of-Vee + right-of-Vee). Every IF-∗ above carries two grounding promises, not one: - a static promise (contract/type signature) — left-of-Vee, grounded by

`reconcile` + schema/model-check against the exposer and consumer; - a dynamic promise (observed runtime behavior) — right-of-Vee, grounded by the runtime oracle (`ACT-Runtime`, §7.1), the 4th fact source.

Integration = grounding the dynamic promise — no new mechanism. An ungrounded dynamic promise is `gap:integration-mismatch` (§10). Failure localizes to the interface's own promise: components A and B keep their groundings (non-monotonicity dissolved). "A was wrong" = an under-specified promise superseded → a diachronic Generation change, not a synchronic retraction. This pattern generalizes to Behaviors (static spec + dynamic trace). Hence integration is no longer deferred to a right-of-Vee phase — it is ordinary promise-discharge inside the one ROUND.

---

7. Actors (5) · Layers (4+1) · Constraints

7.1 Actors

id	name	type	connects via
`ACT-Modeler`	Modeler	human	`consumes`→CLI/MCP · `triggers`→B1/B2/B4/B5
`ACT-Target`	Target System	system (sensed reality)	source of all A7 evidence; `triggers`→B3
`ACT-LLM`	LLM service	external	`consumes`←LLMProviderProtocol
`ACT-Auditor`	Auditor	human	`triggers`→B6/B7 · `consumes`→Artifacts
`ACT-Downstream`	Downstream / arch-agent	system	`consumes`→GraphExport (N1/N2)
`ACT-Runtime`	Runtime / composition-execution	system (sensed reality)	4th fact-source oracle; grounds dynamic (right-of-Vee) interface + behavior promises; `triggers`→B5/B7 · source of executable A7 evidence

Four fact-source oracles (all VoI-arbitrated): code-scan (`ACT-Target`), human-dialogue (`ACT-Modeler`/`ACT-Auditor`), model-self (the graph reading itself), and runtime/composition-execution (`ACT-Runtime`). `ground` is mixed per-axis: structural axes → symbolic, semantic axes → neural.

7.2 Layers

id	name	order	contains	allowed-dep-direction
`LAY-Substrate`	Substrate	0	SourceScanner, PatternDetector, Seeder, FeedbackJournals	upward only

id	name	order	contains	allowed-dep-direction
LAY-Kernel	Kernel	1	13 model components	→ Substrate
LAY-Projection	Projection	2	Slicer, ProjectorRegistry, Renderers, Viewer	→ Kernel
LAY-Adaptation	Adaptation	3	LessonStore, RuleTuner	→ Kernel (via journals)
LAY-CrossCutting	Cross-cutting	—	LLMProvider/Router	any (policy-bounded)

7.3 Constraints

id	name	verification class	scope (A4)	predicate
CON-existence	existence rules	existence → gap:missing	all kinds	required promise present
CON-prohibition	prohibition rules	prohibition → gap:violation	relationships/layers	forbidden edge absent
CON-parametric	budget/fitness	parametric → aggregate-verified	budgets/SLOs	fitness function ≤ bound
CON-pure-state	ModelGraph pure state	prohibition	ModelGraph	no logic in state (anti-god-object)
CON-N2	NN-compilability	existence (machine-evaluable)	ModelGraph export	lossless axis-typed graph round-trips

---

8. I/O-entity family (first-class, owned)

Every output is a modeled entity with `owner = <producing entity>`, an inbound `produces` (A5) edge, and hierarchical ownership = the producer's A1↑ **`contains`** chain up to the System. Reuses the lifecycle layer's existing `owned_paths` / stale invalidation.

id	kind	description	owner (producer)	ownership chain (A1↑)
ART-∗	Artifact	Rendered docs, diagrams, viewers, dashboards, code, graph exports.	Renderers / Viewer / CodeGenerator	→ Projection/mdd layer → openAG
JRN-gates	Journal	gate outcomes (GateEvent).	FeedbackJournals	→ Substrate → openAG
JRN-drift	Journal	drift snapshots.	FeedbackJournals	→ Substrate → openAG
JRN-test	Journal	test_results batches.	FeedbackJournals	→ Substrate → openAG
JRN-llm	Journal	llm_calls records.	FeedbackJournals	→ Substrate → openAG

id	kind	description	owner (producer)	ownership chain (A1↑)
WO-*	WorkOrder	fill_promise:<entity>:<attr> intents.	B2 Authoring (save step)	→ WalkEngine → Kernel → openAG
JOB-*	Job	queued→running→…→succeeded tracking.	BudgetGovernor / Kernel	→ Kernel → openAG
PROP-*	Proposal	ModelPatch + Provenance.	authoring expander	→ ExpanderRegistry → Kernel → openAG
NOTE-*	Note/CommentStub	promise-slot comments.	Modeler (note step)	→ WalkEngine → Kernel → openAG
LSN-*	Lesson	distilled, deduplicated insights.	LessonStore	→ Adaptation → openAG
GEN-*	Generation	published model revisions.	GenerationStore	→ Kernel → openAG
DEC-*	Decision (ADR)	design decisions with rationale.	ExpansionKernel / Modeler	→ Kernel → openAG

Ownership query is answerable at any level: "who owns JRN-drift?" → FeedbackJournals (direct) → Substrate layer → openAG (transitive A1↑). Each I/O entity also appears as a node in the graph export, so output provenance is itself NN-compilable supervision (strengthens C-N2).

---

9. Promise & Evidence nodes (first-class export nodes)

kind	id convention	required attributes (axis)	notes
Promise	{owner}#p{index}	attribute · content · owner_entity · delivered_from · verified_by · status · gap_type · provenance · axis · valence	v3 PromiseSpec + 2 openAG fields; has-promise edge from owner (A7)
Evidence	ev:{sha1(uri)[:8]}	uri · type (static/observational/executable) · digest · verifies-target	deduped + promoted; verified-by edge (A7)

Risk is intrinsic to every Promise — not a new entity. Each promise carries a latent risk = P(won't ground) × consequence, which collapses into the existing VoI machinery: consequence is the value term, P(won't ground) is the confidence term — no new math. Estimation seam: consequence → neural, P → σ + neural. Mitigation reuses existing ops: reduce = ground, transfer = re-bind (reconcile + σ), accept = a gate-L2 waiver → permanent gap. The risk register is a VIEW over the gap register sorted by consequence, surfaced at gate L2 — not stored state.

---

10. Gap taxonomy (sourced from the 3 verification classes)

gap	source class	meaning
`gap:ungrounded`	existence	node has no A7 evidence
`gap:missing`	existence	required promise absent
`gap:unrealized`	existence	capability with no `realizes`
`gap:orphan`	existence	component with no `realizes`-back
`gap:dangling`	existence	interface missing exposer/consumer
`gap:disconnected`	existence	actor with no consumes/triggers
`gap:violation`	prohibition	forbidden edge/state present
`gap:unevaluable`	prohibition	constraint lacks machine-evaluable predicate
`gap:ambiguous-identity`	parametric	θ_low ≤ σ < θ_high at identity bind
`gap:budget-exhausted`	parametric	discharge deferred by widening (free T1 auto-drains; overflow lands here)
`gap:trivial`	parametric (VoI)	neural triviality judgment — promise not worth discharging
`gap:low-confidence`	parametric (VoI)	mixed σ+neural confidence below floor (`gap:confidence-ceiling` = stuck at a ceiling)
`gap:integration-mismatch`	existence (runtime)	interface/behavior dynamic (right-of-Vee) promise ungrounded by the runtime oracle

VoI/economic seam: triviality → neural, confidence → mixed, cost-floor → symbolic. `gap:trivial` distinguishes a legitimate terminal non-gap (plain attribute) from an open typed-gap.

Maps to v3 `gap_type` (`G-DELIVERY`, `G-VERIFY`) + `status: typed-gap`.

---

11. Reconciliation appendix — zero-loss crosswalk

openAG attribute → existing AMS schema `$defs` field, proving no prior effort is lost.

openAG attribute-promise	AMS schema field (`$defs`)	notes
intent	`intent` (base)	identical
success_criteria	`success_criteria`	capability/behavior/interface
failure_modes	`failure_modes` (+ `failure_mode_obj`)	identical
trade_offs	`trade_offs` (+ `trade_off_obj`)	identical
owner	`owner`	component/data/layer
dependencies_rationale	`dependencies_rationale`	component
verification	`verification` (+ `verification_ref_obj`)	identical
slos	`slos` (+ `slo_obj`)	interface/component
maturity	`maturity` (enum)	proposal…deprecated
layer	`layer`	component

openAG attribute-promise	AMS schema field (`$defs`)	notes
subkind	`subkind`	interface (http/cli/message/rpc/library)
mission/F0	(new on System)	openAG adds at L0
axis	(net-new)	openAG extension
valence	(net-new)	openAG extension
promise.status	`PromiseStatus` enum	open/delivered/verified/typed-gap
promise.gap_type	`gap_type`	non-null iff typed-gap
I/O entities	`data/resource/requirement/decision` kinds + lifecycle `owned_paths`	openAG promotes to first-class owned family

Relationship types: all 23 AMS `RelationType` values are mapped to the 7 axes (design §3.1). No relationship type is dropped.

---

12. Growth Frontier (why this is gen-0, not final)

The entity set is open by construction (invariant #5, gap-honesty). Gen-1+ entities will be discovered from:

1. L3+ fan-out — the §3 non-atomic branches may spawn further sub-functions and support components as expanders are implemented.
2. **`gap:deferred`** v2 items — A6 Lifecycle axis, behavior flavors + operational modes, trade-off/MOE Pareto, federated multi-package growth, N2 compiler/trainer + `CAP:mdd` neural backend.
3. Meta-closure — running openAG on openAG (as AMS self-models via `capabilities.v3.yaml`) discovers entities not hand-authored here.
4. Per-discharge nodes — `Promise`, `Evidence`, and every §8 I/O entity are generated per round — unbounded by construction.

Declaring this model "final" would violate gap-honesty. It is the seed from which the fixpoint grows.

---

13. Companion documents

- `2026-10-04-openAG-graph-native-engine-design.md` — formal core, axes, operator, invariants, attribute algebra, critique + 4 solutions, N1/N2.
- `2026-10-04-openAG-pipelines.md` — B-ROUND + B1–B7 as pipelines, comparison matrix vs. the 5 existing AMS pipelines, operator crosswalk.